



PROJECT NAME: EduGenie - Google Gemini Powered Learning Assistant
TEAM ID: SWTID-2026-8551

A PROJECT REPORT

Submitted by

NAME	REGISTER NUMBER
Harini. S	222503889
Banu. C	222503887
Kavijasri. S	222503893

In partial fulfilment for the award of the degree

of

BACHELOR

OF

COMPUTER SCIENCE

SRIDEVI ARTS AND SCIENCE COLLEGE
KRISHNAPURAM, PONNERI-601 204

SEPTEMBER-2026

EduGenie – Google Gemini Powered Learning Assistant is a lightweight AI-powered educational web application designed to support students with common learning tasks. It provides a single interface for asking questions, understanding topics, generating quizzes, summarizing educational content, and receiving structured learning recommendations.

The project uses a FastAPI backend with an HTML/CSS frontend and Google Gemini through the Google GenAI Python SDK. The application is organized into separate Python modules so that each

learning function can be developed and maintained independently.

Key capabilities:

- Ask questions and receive concise, student-friendly answers.
- Explain difficult topics in a simple and structured way.
- Generate three multiple-choice questions with four options each.
- Summarize long educational passages into important points.
- Generate beginner-to-advanced learning paths with practice/resource suggestions.

SCENARIOS

- A student asks a question such as “What is Python?” and receives an AI-generated explanation.
- A learner enters a topic and uses the quiz option to create practice questions.
- A student studying a new subject requests a learning path that progresses from beginner to advanced concepts.

The project is intended to make self-learning more accessible through a simple browser-based interface.

PREREQUISITES

1. Python 3.10+

- Python is used to implement the backend and AI modules.
- The development environment used for this project is Python 3.10.11.
- Python packages are installed inside a virtual environment (venv).

2. FastAPI Framework

- FastAPI provides the backend application and REST-style POST endpoints.
- It connects the web interface with the individual AI modules.

3. HTML & CSS

- HTML provides the web interface and input controls.
- CSS provides layout, spacing, buttons, cards, and responsive styling.

4. Google Gemini API

- A Google AI Studio API key is used for Gemini-powered generation.
- The API key is stored in a local .env file and is not placed directly in the source code.

5. Uvicorn

- Uvicorn runs the FastAPI application locally.

6. Jinja2 and python-dotenv

- Jinja2 renders the HTML template through FastAPI.
- python-dotenv loads the Gemini API key from the .env file.

ARCHITECTURE AND FOLDER STRUCTURE

The application follows a modular client-server architecture. The browser sends a request to FastAPI, FastAPI routes the request to the appropriate Python module, and the module communicates with Gemini to generate the required learning content.

Folder Architecture

File / Folder	Purpose
main.py	FastAPI application and API endpoints
qna.py	Question-answering logic
explanation_module.py	Topic explanation logic
quiz_module.py	Quiz generation logic
summary_module.py	Text summarization logic
learning_path.py	Learning recommendation logic
templates/index.html	Frontend HTML interface
static/style.css	Frontend styling
.env	Local Gemini API key configuration
requirements.txt	Python dependencies

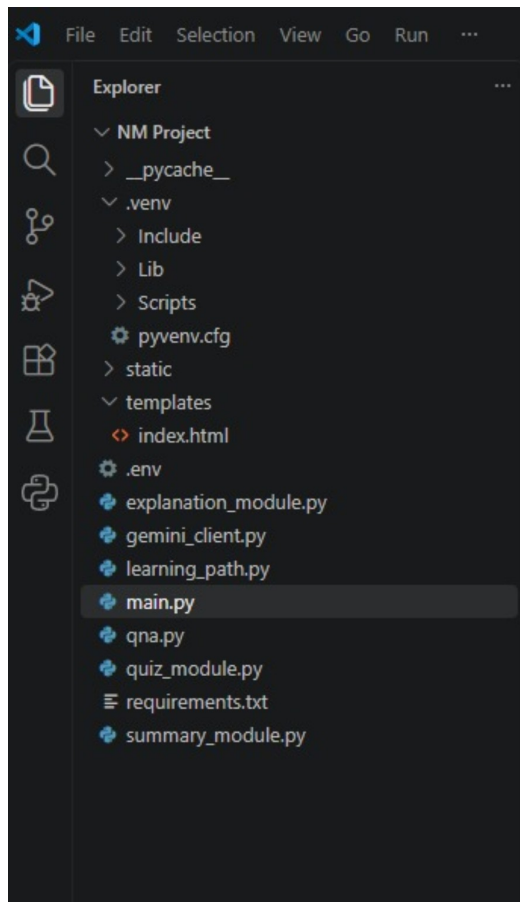


Fig. Folder structure of the EduGenie development project

MILESTONE 1: MODEL SELECTION AND ARCHITECTURE

Activity 1.1: AI Integration

EduGenie uses Google Gemini as its cloud-based generative AI service. The Google GenAI Python SDK is initialized using the API key stored in the .env file. Each learning module sends a task-specific prompt and returns the generated text to the FastAPI endpoint.

The current implementation uses a modular client pattern. This keeps the API configuration separate from the learning functions and makes it easier to maintain the application.

Core architecture flow:

- User → HTML/CSS frontend
- Frontend → FastAPI POST endpoint
- FastAPI → selected Python module
- Python module → Google Gemini
- Gemini response → FastAPI → browser result area

The modular structure follows the organization of the supplied project model while reflecting the actual implementation used for this project.

MILESTONE 2: CORE FUNCTIONALITIES DEVELOPMENT

Activity 2.1: Module Implementation

1. QnA Module

The QnA module accepts a student's question and builds an educational prompt for Gemini. The prompt asks the model to answer clearly and accurately and to keep the explanation suitable for a student. The returned text is sent to the frontend through the /qa endpoint.

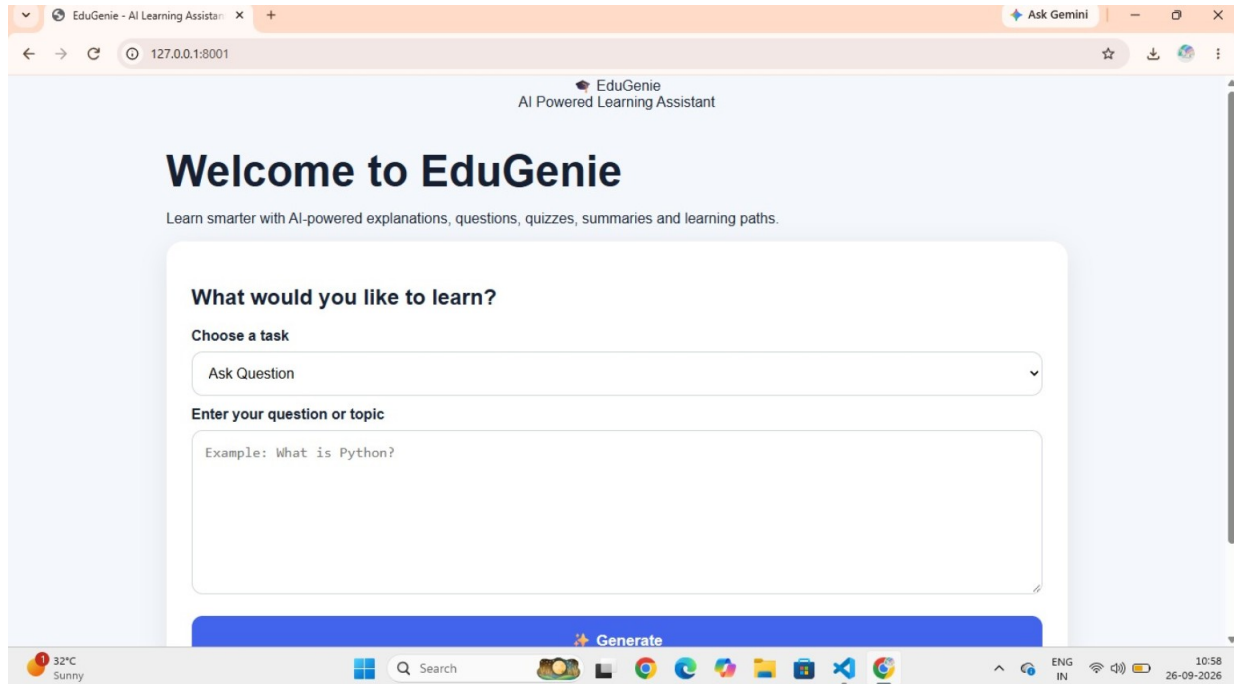


Fig. EduGenie web interface with the Ask Question task

The interface provides a task selector and a text area where the learner enters a question or topic. The Generate button sends the request to the backend.

2. EXPLANATION MODULE

The Explanation module receives a topic and asks Gemini to explain it in a simple educational format. The prompt is designed to produce a definition, main points, a simple example, and a short conclusion. This structure helps students understand unfamiliar concepts without requiring a long technical explanation.

The module is connected to the /explain endpoint in FastAPI. Errors from the AI service are returned as readable error messages so that the application can continue running instead of crashing.

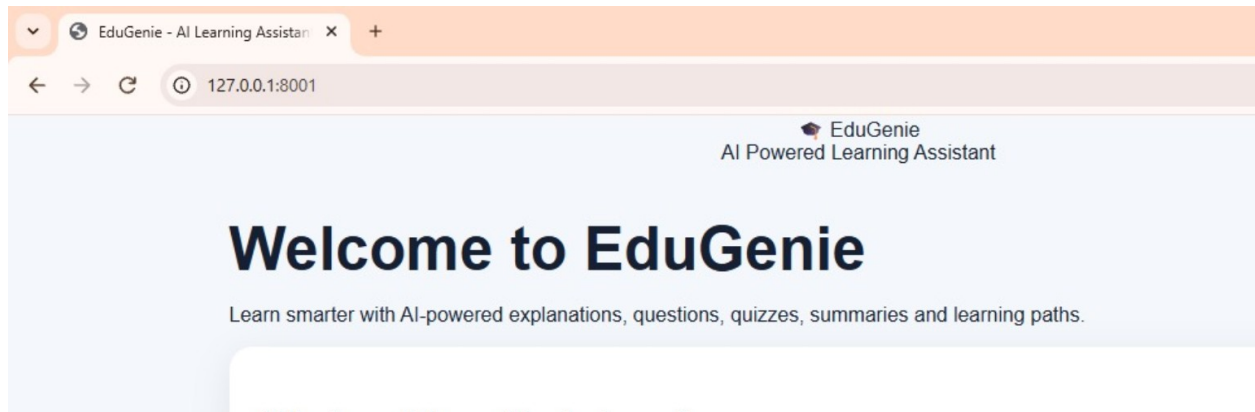


Fig. EduGenie interface showing the learning assistant layout

3. QUIZ MODULE

The Quiz module creates a short practice quiz from a topic or supplied educational content. The prompt requests exactly three multiple-choice questions and four options (A, B, C, D) for each question, together with the correct answer.

This module is useful for self-assessment because the learner can use the generated questions to check understanding after studying a topic.

The quiz request is handled by the /quiz FastAPI endpoint and the generated result is returned to the browser.

Quiz generation flow:

- Receive topic or content from the frontend.
- Construct a structured Gemini prompt.
- Generate three MCQs with four options.
- Return the generated quiz text to the result section.

4. SUMMARY MODULE

The Summary module accepts a long passage and asks Gemini to produce a concise version suitable for a college student. The prompt requests the main idea, important points, and a short conclusion.

This feature is designed for quick revision. It reduces lengthy educational content while retaining the key ideas needed for understanding.

The summary operation is exposed through the /summarize endpoint.

Summary workflow:

- User enters or pastes educational text.
- Frontend sends the text to FastAPI.
- FastAPI calls summarize_text().
- Gemini generates a concise structured summary.
- The result is displayed below the input area.

5. LEARNING PATH MODULE

The Learning Path module generates a structured plan for a selected topic. It asks Gemini to organize learning into beginner, intermediate, and advanced levels and to include explanations and useful practice/resource ideas.

This module helps a learner understand what to study first and how to progress toward more advanced concepts. The functionality is exposed through `/learn/recommendations`.

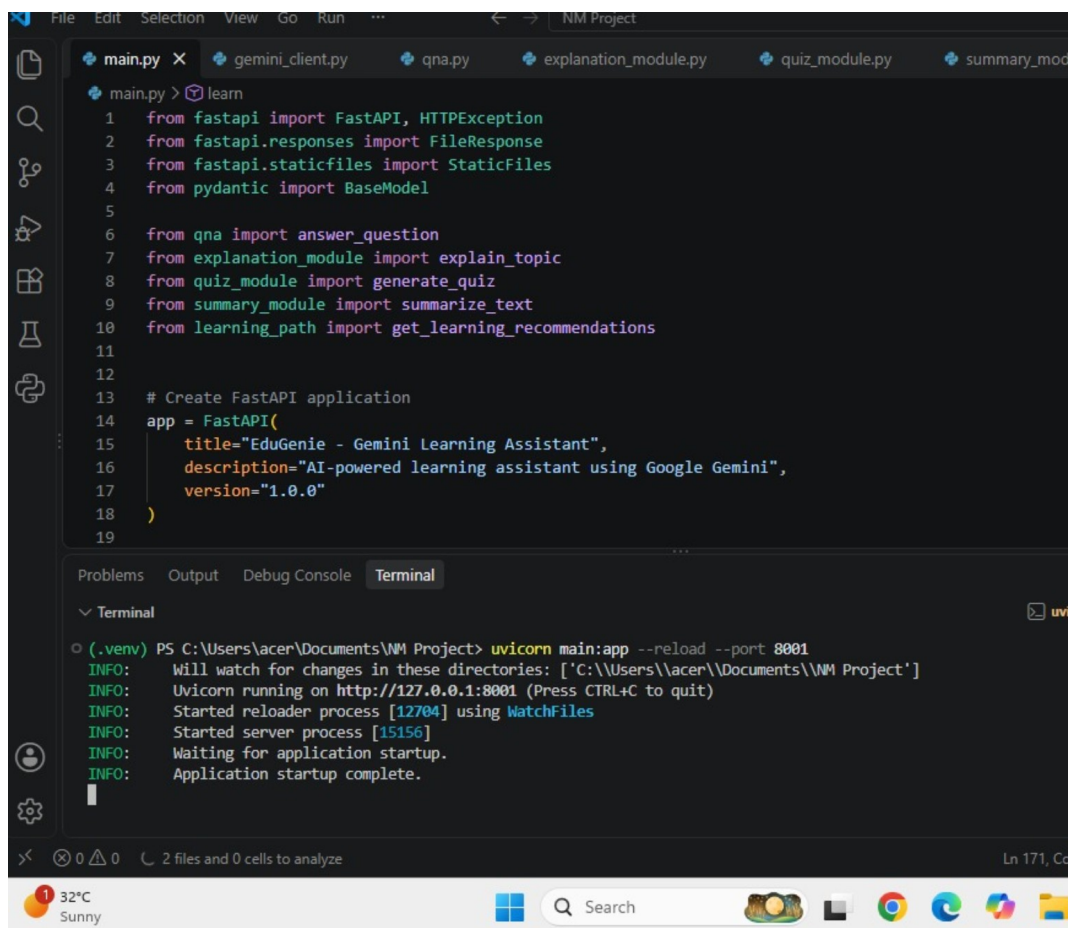
Learning path structure:

- Beginner level – fundamental concepts and introductory practice.
- Intermediate level – deeper concepts and applied exercises.
- Advanced level – advanced concepts, projects, and further resources.

ACTIVITY 2.2: BACKEND API WITH FASTAPI

FastAPI acts as the central backend layer. The application defines separate endpoints for each learning function and connects each endpoint to its corresponding module.

Method	Endpoint	Function
GET	/	Loads the EduGenie web interface
POST	/qa	Question answering
POST	/explain	Topic explanation
POST	/quiz	Quiz generation
POST	/summarize	Text summarization
POST	/learn/recommendations	Learning path generation



The screenshot displays a code editor with the following Python code in `main.py`:

```
1 from fastapi import FastAPI, HTTPException
2 from fastapi.responses import FileResponse
3 from fastapi.staticfiles import StaticFiles
4 from pydantic import BaseModel
5
6 from qna import answer_question
7 from explanation_module import explain_topic
8 from quiz_module import generate_quiz
9 from summary_module import summarize_text
10 from learning_path import get_learning_recommendations
11
12
13 # Create FastAPI application
14 app = FastAPI(
15     title="EduGenie - Gemini Learning Assistant",
16     description="AI-powered learning assistant using Google Gemini",
17     version="1.0.0"
18 )
19
```

The terminal window shows the command `uvicorn main:app --reload --port 8001` and its output:

```
INFO: Will watch for changes in these directories: ['C:\\Users\\acer\\Documents\\NM Project']
INFO: Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)
INFO: Started reloader process [12704] using WatchFiles
INFO: Started server process [15156]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Fig. FastAPI main application and module imports

MILESTONE 3: FRONTEND DEVELOPMENT

Activity 3.1: Build Web Interface

The frontend is implemented using HTML and CSS and is rendered through the FastAPI/Jinja2 template system. The interface is designed as a simple single-page learning assistant so that students can select a task and enter their content without navigating through multiple pages.

- Task dropdown: Ask Question, Explain Topic, Generate Quiz, Summarize, and Learning Path.
- Large text area for entering a question, topic, or passage.
- Generate button for submitting the selected task.
- Result area for displaying the generated response.

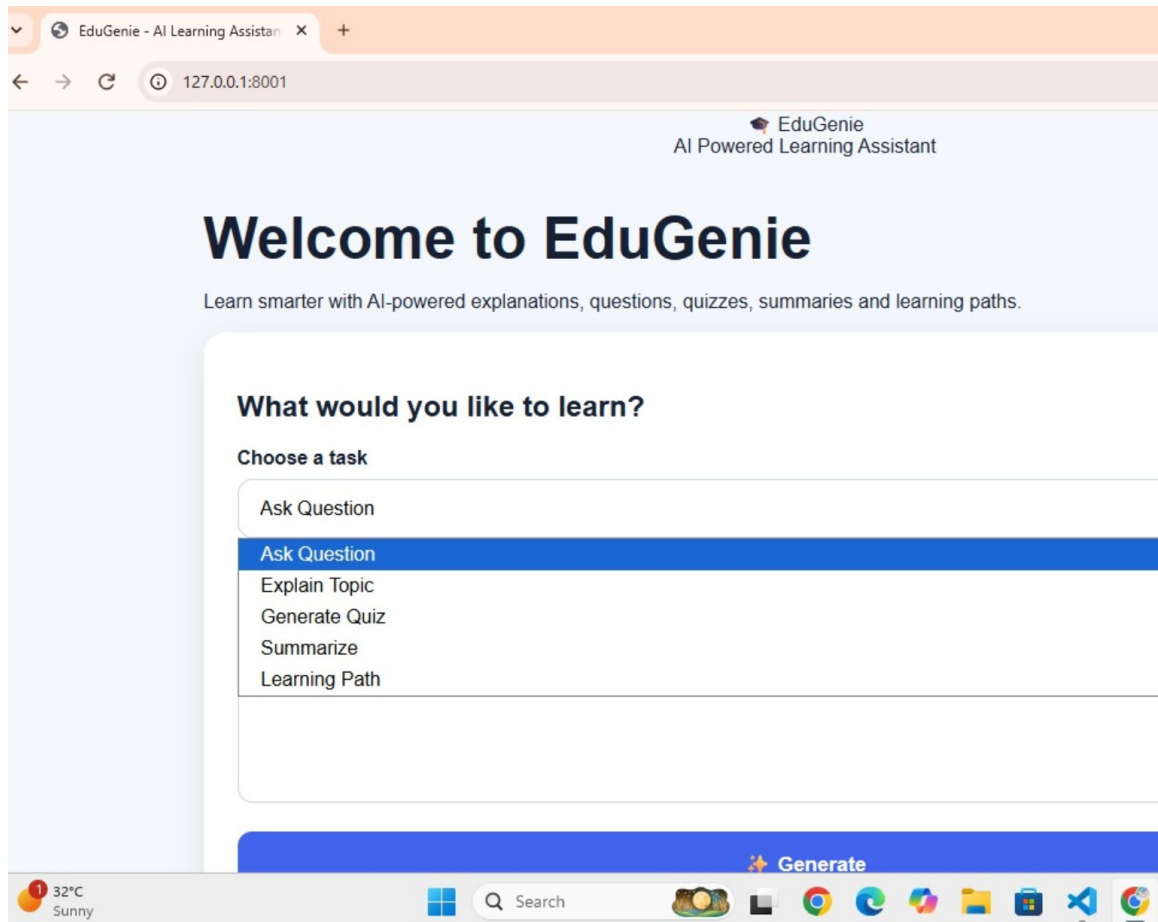


Fig. EduGenie frontend task selection and input interface

Activity 3.2: Live Integration

The browser sends a POST request to the appropriate FastAPI endpoint. The returned result is then displayed on the same page, giving the user a simple interactive learning experience.

FRONTEND INTEGRATION AND USER EXPERIENCE

The screenshots below show the working EduGenie interface at the local development address. The home page presents the project title, a short description, the task selector, and the input area.

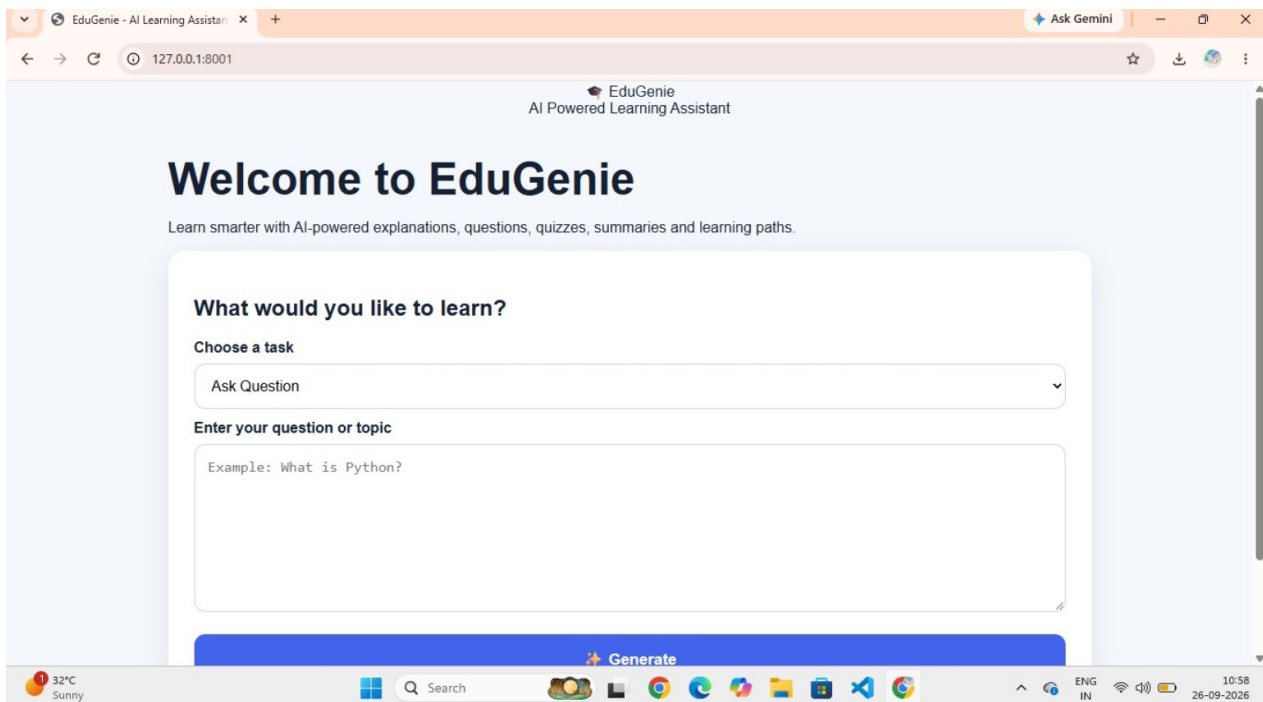


Fig. EduGenie home page with task dropdown

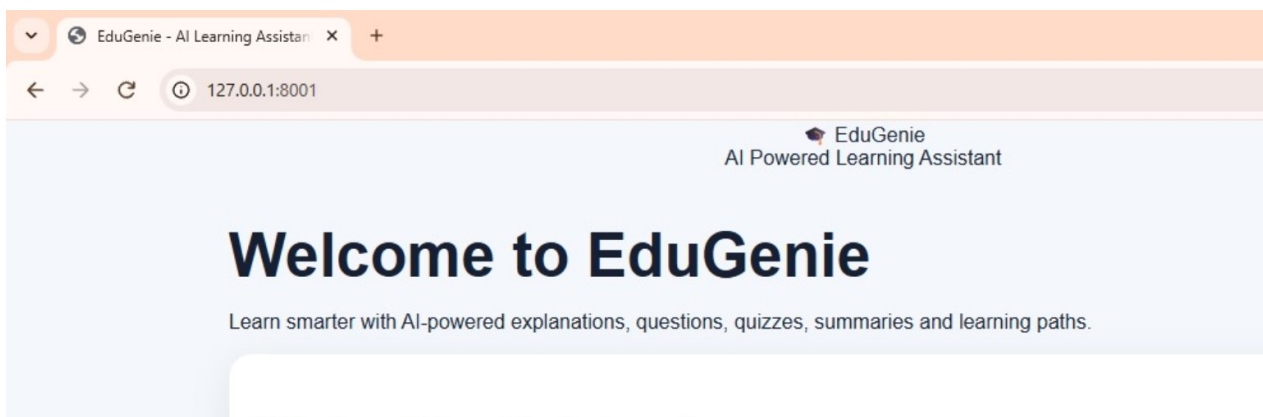


Fig. EduGenie page header and learning assistant layout

MILESTONE 4: DEPLOYMENT

Activity 4.1: Run Locally

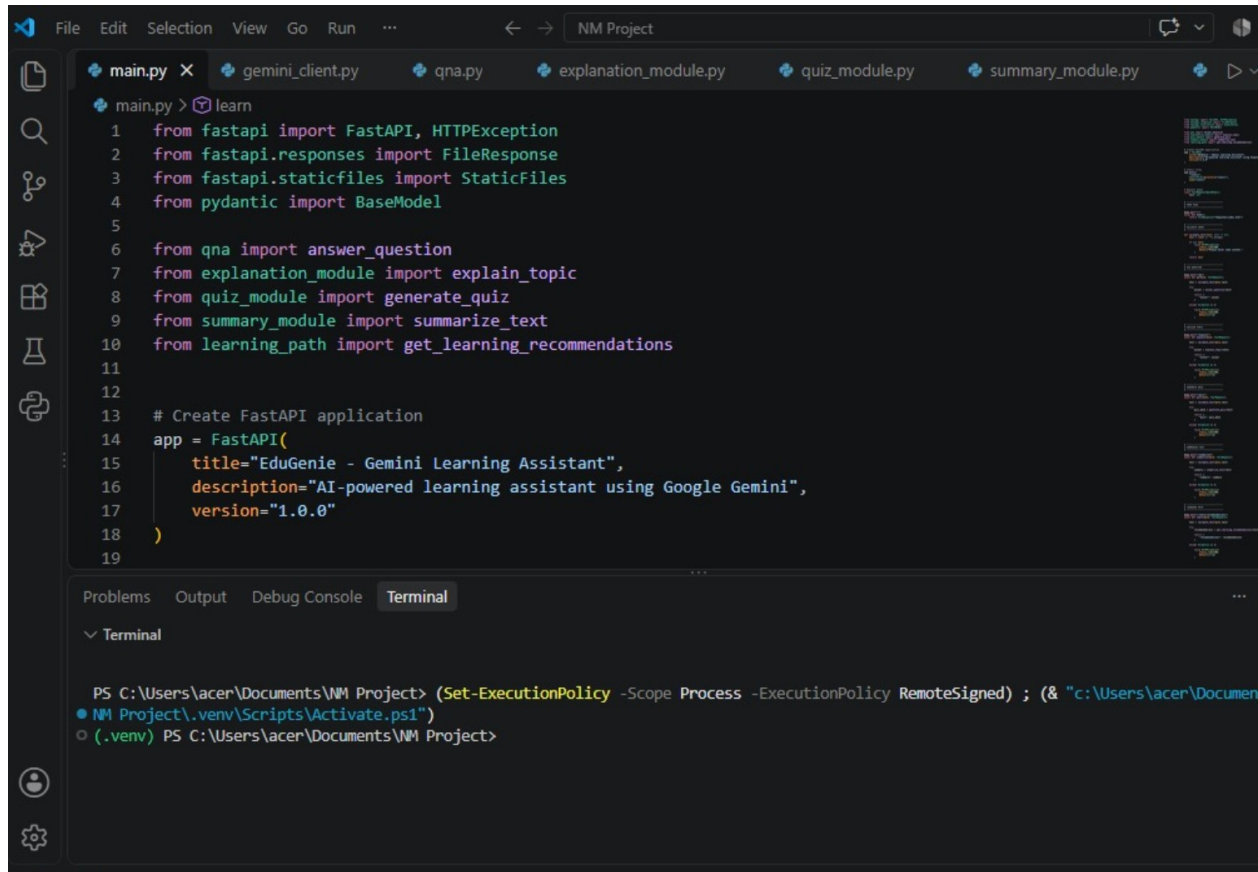
The application is run locally using Uvicorn. During development, the server is started with the reload option so that source-code changes can be reflected without manually restarting the server each time.

Command used:

```
uvicorn main:app --reload --port 8001
```

The application is accessed through the local browser address:

```
http://127.0.0.1:8001
```



```
File Edit Selection View Go Run ...
main.py x gemini_client.py qna.py explanation_module.py quiz_module.py summary_module.py
main.py > learn
1 from fastapi import FastAPI, HTTPException
2 from fastapi.responses import FileResponse
3 from fastapi.staticfiles import StaticFiles
4 from pydantic import BaseModel
5
6 from qna import answer_question
7 from explanation_module import explain_topic
8 from quiz_module import generate_quiz
9 from summary_module import summarize_text
10 from learning_path import get_learning_recommendations
11
12
13 # Create FastAPI application
14 app = FastAPI(
15     title="EduGenie - Gemini Learning Assistant",
16     description="AI-powered learning assistant using Google Gemini",
17     version="1.0.0"
18 )
19
Problems Output Debug Console Terminal
Terminal
PS C:\Users\acer\Documents\NM Project> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "c:\Users\acer\Documents\NM Project\venv\Scripts\Activate.ps1")
(.venv) PS C:\Users\acer\Documents\NM Project>
```

Fig. Uvicorn server running successfully with Application startup complete

ACTIVITY 4.2: FUNCTIONAL TESTING

The application was checked through its browser interface and local FastAPI server. The available functions are represented in the task selector and mapped to the corresponding backend endpoints.

- Ask Question – sends a question to /qa.
- Explain Topic – sends a topic to /explain.
- Generate Quiz – sends content to /quiz.
- Summarize – sends text to /summarize.
- Learning Path – sends a topic to /learn/recommendations.

Development evidence:

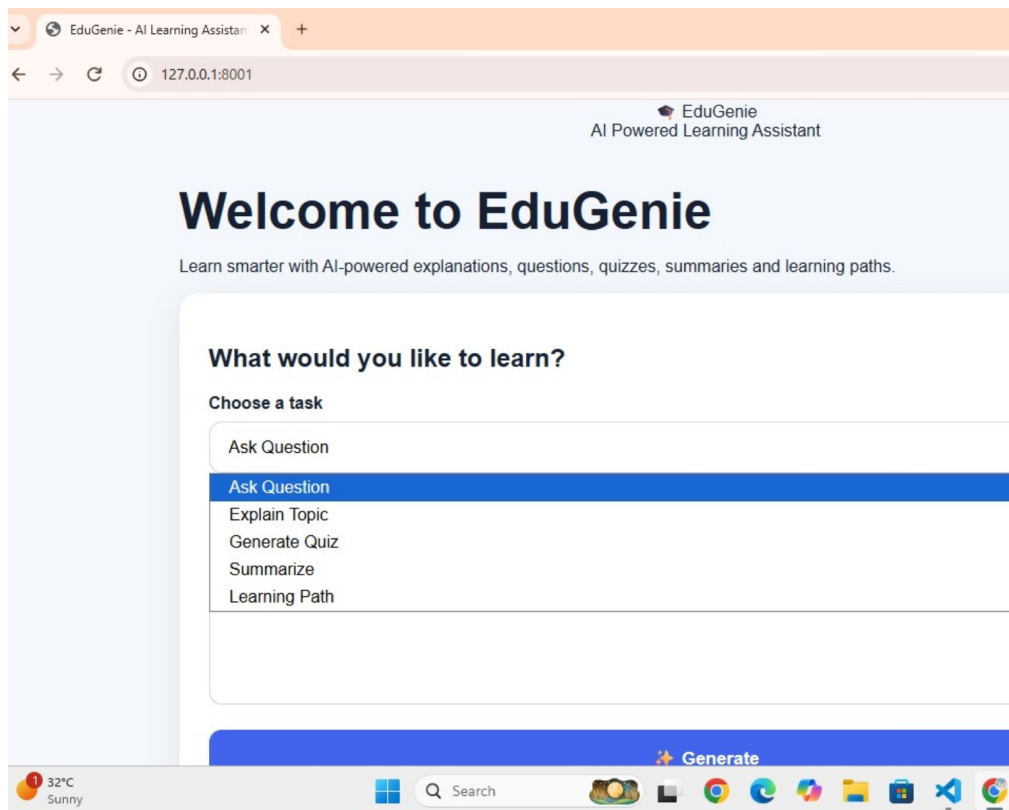


Fig. EduGenie browser interface used for functional testing

The modular project structure and running server were also verified during development.

PROJECT IMPLEMENTATION RESULT

The completed implementation provides a lightweight browser-based learning assistant with a FastAPI backend and modular Gemini-powered learning functions. The project structure separates frontend, backend routing, AI prompts, configuration, and styling.

The implementation demonstrates the integration of:

- Python and FastAPI for web application development.
- Jinja2 for HTML template rendering.
- HTML/CSS for the user interface.
- Google Gemini through the Google GenAI SDK for generative learning features.
- Uvicorn for local ASGI server execution.
- Environment variables for keeping the API key outside the source files.

The supplied screenshots show the final browser interface, backend code, local server, and project folder organization.

CONCLUSION

EduGenie has been developed as an AI-powered educational assistant that combines a simple web interface with cloud-based generative AI. It supports question answering, topic explanation, quiz generation, summarization, and structured learning recommendations.

The project demonstrates practical integration of a modern Python web framework with generative AI services. Its modular design makes the individual learning functions easier to maintain and extend. The application also provides a simple user experience in which the learner selects a task, enters content, and receives the generated result.

FUTURE ENHANCEMENTS

- Voice-based questions and spoken responses.
- Multilingual learning support.
- Mobile application support.
- Student progress tracking and dashboards.
- Gamified learning features such as badges and streaks.
- Adaptive learning paths based on learner performance.
- Image and PDF input for doubt solving and summarization.
- Teacher/parent dashboards and LMS integration.

Submitted by

Harini. S & Team

B.Sc. Computer Science

Sri Devi Arts and Science College

September 2026